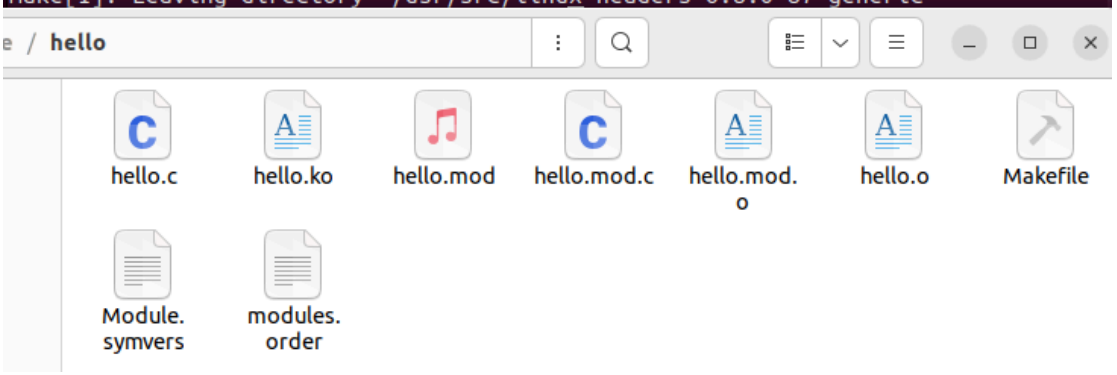
Setting up the Environment: Verify if kernel headers and development tools are installed on the system. (Command: uname -r and sudo apt install build-essential linux-headers-$(uname -r)). Provide your screenshot. (5 pts)

```
wesannn@wesannn-virtual-machine:~$ uname -r
6.8.0-87-generic
wesannn@wesannn-virtual-machine:~$ sudo apt install build-essential linux-header
s-$(uname -r)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.9ubuntu3).
linux-headers-6.8.0-87-generic is already the newest version (6.8.0-87.88~22.04.
1).
linux-headers-6.8.0-87-generic set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Prepare a simple “Hello World” kernel module. You are asked to write a simple “Hello World” kernel module and a Makefile to compile your kernel module. (10 pts)

```
wesannn@wesannn-virtual-machine:~/hello$ make
make -C /lib/modules/6.8.0-87-generic/build M=/home/wesannn/hello modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-87-generic'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04
.2) 12.3.0
  You are using:           gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.2) 12.3.0
  CC [M]  /home/wesannn/hello/hello.o
  MODPOST /home/wesannn/hello/Module.symvers
  CC [M]  /home/wesannn/hello/hello.mod.o
  LD [M]  /home/wesannn/hello/hello.ko
  BTF [M] /home/wesannn/hello/hello.ko
Skipping BTF generation for /home/wesannn/hello/hello.ko due to unavailability o
f vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-87-generic'
```



Loading and unloading the module. After compiling your module, you should use commands (i.e., insmod and rmmod) to load and unload your module. Moreover, you need to verify the module status by checking the kernel log (command: dmesg). (10 pts)

```
wesannn@wesannn-virtual-machine:~/hello$ sudo insmod hello.ko
wesannn@wesannn-virtual-machine:~/hello$ dmesg | tail
dmesg: read kernel buffer failed: Operation not permitted
wesannn@wesannn-virtual-machine:~/hello$ sudo dmesg | tail
[   12.643955] audit: type=1400 audit(1763636227.573:56): apparmor="DENIED" oper
ation="capable" class="cap" profile="/usr/sbin/cupsd" pid=1173 comm="cupsd" capa
bility=12  capname="net_admin"
[   13.457392] loop8: detected capacity change from 0 to 8
[   14.113407] audit: type=1400 audit(1763636229.042:57): apparmor="STATUS" oper
ation="profile_replace" profile="unconfined" name="/usr/lib/snapd/snap-confine"
pid=1339 comm="apparmor_parser"
[   14.124079] audit: type=1400 audit(1763636229.053:58): apparmor="STATUS" oper
ation="profile_replace" profile="unconfined" name="/usr/lib/snapd/snap-confine//
mount-namespace-capture-helper" pid=1339 comm="apparmor_parser"
[   29.977677] rfkill: input handler disabled
[   38.275203] input: VMware DnD UInput pointer as /devices/virtual/input/input6
[  160.423688] workqueue: vmballoon_work [vmw_balloon] hogged CPU for >10000us 4
 times, consider switching to WQ_UNBOUND
[ 1327.187326] hello: loading out-of-tree module taints kernel.
[ 1327.187331] hello: module verification failed: signature and/or required key
missing - tainting kernel
[ 1327.188256] Hello, world!
```

```
wesannn@wesannn-virtual-machine:~/hello$ sudo rmmod hello.ko
wesannn@wesannn-virtual-machine:~/hello$ sudo dmesg | tail
[   13.457392] loop8: detected capacity change from 0 to 8
[   14.113407] audit: type=1400 audit(1763636229.042:57): apparmor="STATUS" oper
ation="profile_replace" profile="unconfined" name="/usr/lib/snapd/snap-confine"
pid=1339 comm="apparmor_parser"
[   14.124079] audit: type=1400 audit(1763636229.053:58): apparmor="STATUS" oper
ation="profile_replace" profile="unconfined" name="/usr/lib/snapd/snap-confine//
mount-namespace-capture-helper" pid=1339 comm="apparmor_parser"
[   29.977677] rfkill: input handler disabled
[   38.275203] input: VMware DnD UInput pointer as /devices/virtual/input/input6
[  160.423688] workqueue: vmballoon_work [vmw_balloon] hogged CPU for >10000us 4
 times, consider switching to WQ_UNBOUND
[ 1327.187326] hello: loading out-of-tree module taints kernel.
[ 1327.187331] hello: module verification failed: signature and/or required key
missing - tainting kernel
[ 1327.188256] Hello, world!
[ 1444.730482] Goodbye, world!
```

Recompiling the kernel to include the module. Students are asked to clone the Linux kernel source and reconfigure the kernel to build the module as a built-in part of the kernel (not a loadable module). Verify the kernel includes the module by checking dmesg. (15 pts)

```
wesannn@wesannn-virtual-machine:~$ uname -r
6.18.0-rc6+
wesannn@wesannn-virtual-machine:~$ sudo dmesg | grep -i hello
[    1.945397] Hello, world!
```

Demonstration: Students must demonstrate that the module loads automatically at boot or via manual insertion (modprobe). Moreover, you should submit screenshots of their compiled module, kernel messages from dmesg, and a short report explaining their process. (10 pts)

一開始確認Kernel版本是 6.8.0-87-generic，以及安裝編譯所需的開發工具、kernel headers，接著建立一個kernel module ”Hello” 並在Makefile內寫

```
obj-m += hello.o

all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

來compile出kernel module “hello.ko”，並使用insmod跟rmmod 來手動insert/remove，可以透過dmesg確認是否成功載入/卸載。

接下來想讓module是在開機後就會自動載入，所以把 hello.c移至drivers/misc內，並修改Kconfig加入:

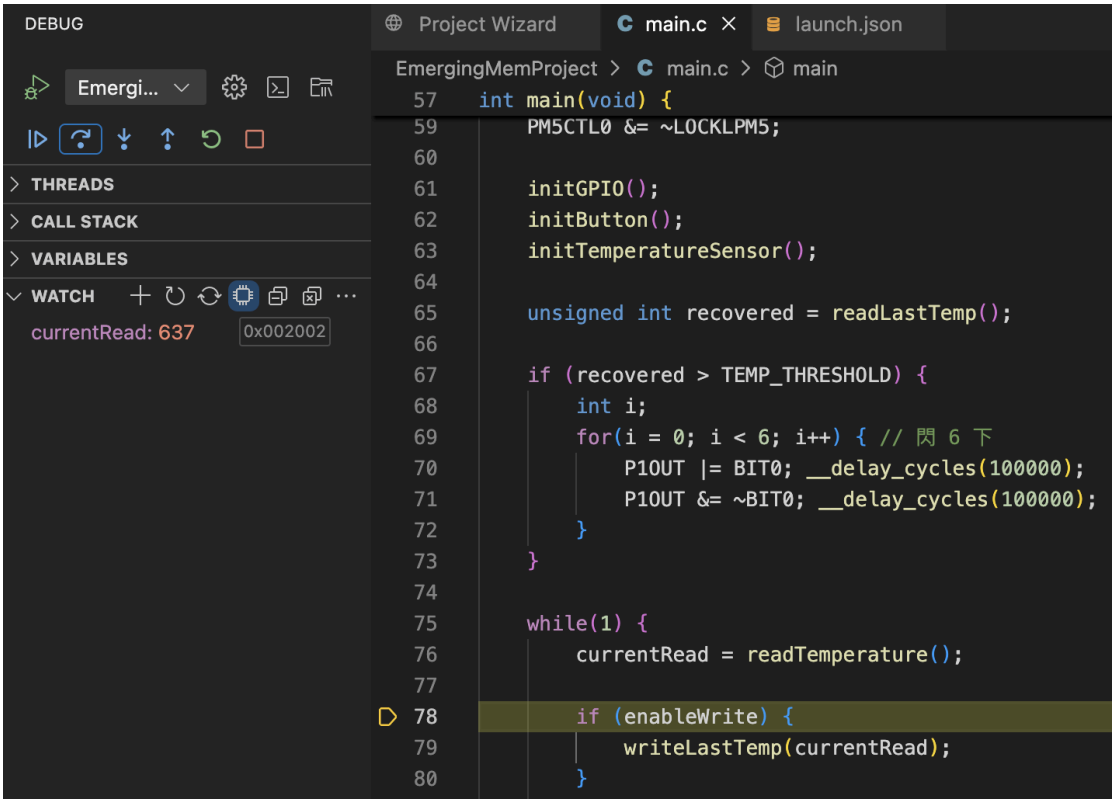
```
config HELLO_MODULE
  bool "Built-in Hello World Module"
  default y
```

，以及修改Makefile加入:

```
obj-$(CONFIG_HELLO_MODULE) += hello.o
```

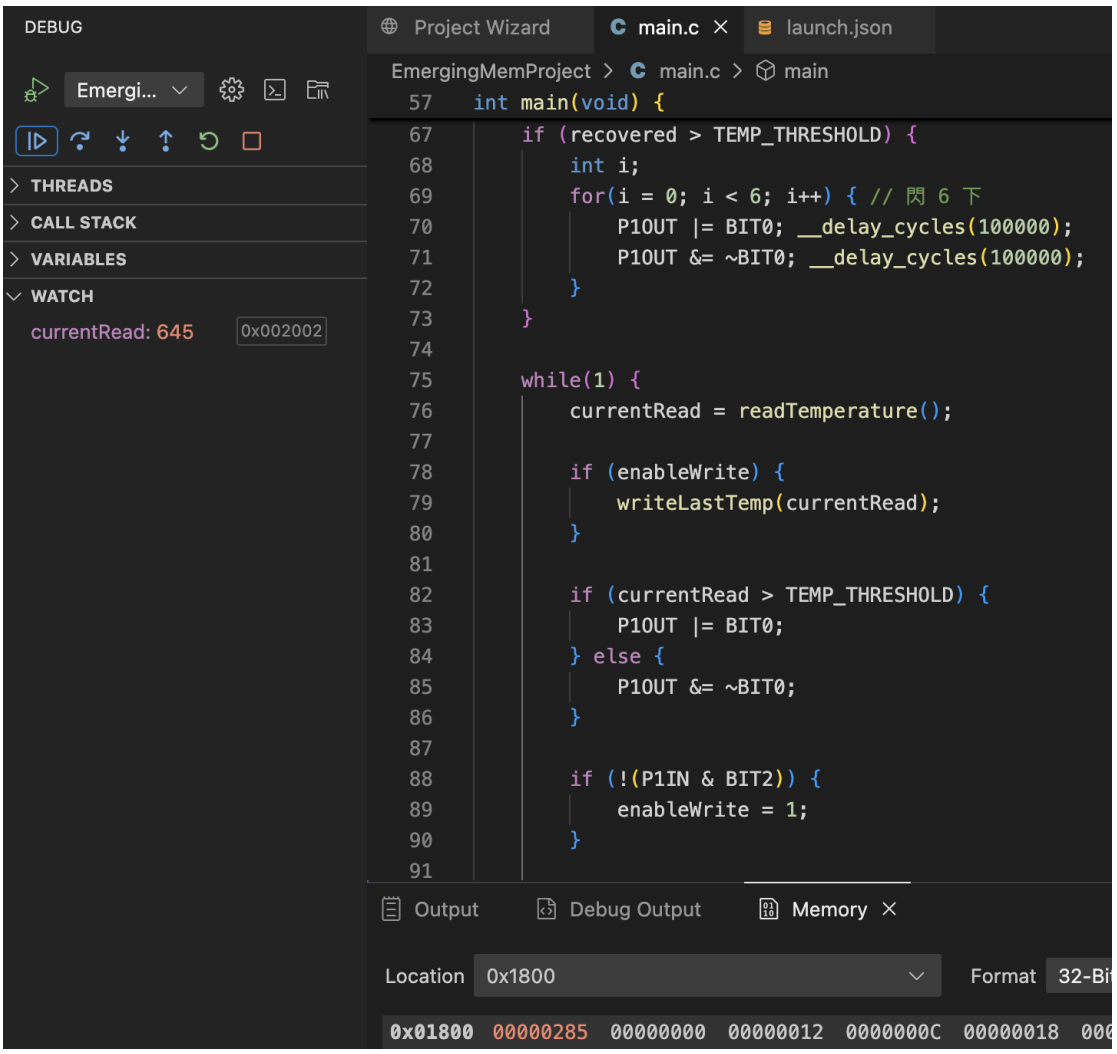
再來執行make menuconfig，在Device Drivers -> Misc devices -> 找到新增的 [*] Built-in Hello World Module，選到它後按”Y”啟用為built-in，最後重新compile Kernel並重開機，確認新版kernel為6.18.0-rc6+，也在dmesg中找到開機後1.9秒有自動執行hello.ko顯示出 “Hello, world!” 代表已成功為 built-in kernel module，並在開機過程中自動執行。

- Setting up the temperature sensor. Students are tasked with configuring the MSP430's ADC12 to collect data from the internal temperature sensor. Implement the function readTemperature() to periodically collect temperature data. (10 pts)



透過 readTemperature() 讀取溫度到變數 currentRead。

- Set up FRAM storage for the temperature data. Ensure that the temperature data collected is written into FRAM after every reading. Students should ensure that each time the system reads a new temperature value, it is stored into FRAM immediately. (10 pts)



執行函式 writeLastTemp() 將讀到的溫度，存到設定好的 FRAM 區間 0x1800 (多了一個 enableWrite 變數是為了透過按鈕處發後才開放寫入，方便斷電後的資料顯示。)

- Modify the system so that after a power loss, the last temperature data stored in FRAM is correctly retrieved and used for further processing. This ensures that no data is lost. Implement a power recovery mechanism where, upon reboot, the system checks FRAM for the last valid temperature value and uses it for processing. (10 pts)

```
unsigned int recovered = readLastTemp();

if (recovered > TEMP_THRESHOLD) {
    int i;
    for(i = 0; i < 6; i++) { // 閃 6 下
        P1OUT |= BIT0; __delay_cycles(100000);
        P1OUT &= ~BIT0; __delay_cycles(100000);
    }
}
```

在開機後，先讀取上次存在 FRAM 的溫度，若是過熱的溫度，會先做 6 次的p1.0 LED 閃爍，若之前的溫度沒有過熱，則不會閃爍。

- Use the temperature data retrieved from FRAM to implement a simple application, such as comparing the current temperature to a set threshold. If the temperature exceeds a specific threshold, trigger an LED on the board to indicate overheating. (10 pts)

```
if (recovered > currentRead) {
    P1OUT |= BIT0;
} else {
    P1OUT &= ~BIT0;
}
```

用 FRAM 存取的溫度 recovered，與讀取溫度到的 currentRead 比較，如果大於則長亮 P1.0 LED燈表示過熱，反之不亮燈。

- Demonstration: Students should test the system by simulating power loss and recovery: Collect temperature data, power off the system, and power the system back on and verify that the stored temperature data is correctly retrieved and used. (10 pts)

DEMO 影片連結：
https://drive.google.com/file/d/1UpVGIPQQkbj1yxSisPDhm5wK9JvEHQT2/view?usp

=drive_link

影片中斷電後仍保留前次最後存入的溫度 646 (0x286)，且有超標 THRESHOLD 635，因此重新接電 debug 後會閃爍 P1.0 LED 燈 6 次，也因為 recovered 溫度超標 currentRead，所以 P1.0 會長亮燈。

並且因為有設置按下 P1.2 按鈕才開放寫入 FRAM 區間，所以重新 debug 後顯示出 currentRead 已經讀了新數值，但是 FRAM 仍保持斷電前的646 (0X286)。

最後附上完整 code:

```
#include <msp430.h>
```

```
#include <msp430fr4133.h>


#define LASTTEMP_ADDR   ((unsigned int*)0x1800)

unsigned int *lastTempPtr = (unsigned int *)0x1800;  // Data FRAM 起始位址


unsigned int currentRead = 0;

#define TEMP_THRESHOLD 635


void initTemperatureSensor() {

   PMMCTL0_H = PMMPW_H;           // 解鎖 PMM 電源管理

   PMMCTL2 |= INTREFEN | TSENSOREN; // [重點] 必須同時開啟 INTREFEN (內部參考電壓) 和 TSENSOREN (溫度感測器開關)

   __delay_cycles(10000); // 給它一點時間穩定

   PMMCTL0_H = 0;                 // 鎖定 PMM


   // --- ADC 設定區 ---

   ADCCTL0 &= ~ADCENC;            // 設定前先 Disable

   ADCCTL0 = ADCSHT_8 | ADCON;    // ADCSHT_8: 256 cycles, 長取樣時間確保讀數準確

   ADCCTL1 = ADCSHP;              // 使用採樣計時器

   ADCCTL2 = ADCRES_1;            // 10-bit 解析度 (0~1023)

   ADCMCTL0 = ADCINCH_12 | ADCSREF_1; // Channel 12 是溫度感測器
```

```
    ADCCTL0 |= ADCENC;
}

unsigned int readTemperature() {
    ADCCTL0 |= ADCENC | ADCSC;     // 開始轉換
    while (ADCCTL1 & ADCBUSY);     // 等待忙碌標誌結束
    return ADCMEM0;                // 回傳結果
}

void initGPIO() { // LED燈 P1.0
    P1DIR |= BIT0; // Set P1.0 to output directionOUT
    P1OUT &= ~BIT0; // Clear P1.0 output latch for a defined power-on
state
}

unsigned char enableWrite = 0;

void initButton() { // P1.2 按鈕
    P1DIR &= ~BIT2;    // P1.2 作輸入
    P1REN |= BIT2;     // 開啟 pull-up/down
    P1OUT |= BIT2;     // pull-up
}

void writeLastTemp(unsigned int v) {
    SYSCFG0 &= ~DFWP;         // unlock Data FRAM
    *lastTempPtr = v;
    SYSCFG0 |= DFWP;          // lock again
}
```

```
unsigned int readLastTemp() {
    return *lastTempPtr;
}

int main(void) {
    WDTCTL = WDTPW | WDTHOLD;
    PM5CTL0 &= ~LOCKLPM5;

    initGPIO();
    initButton();
    initTemperatureSensor();

    unsigned int recovered = readLastTemp();

    if (recovered > TEMP_THRESHOLD) {
        int i;
        for(i = 0; i < 6; i++) { // 閃 6 下
            P1OUT |= BIT0; __delay_cycles(100000);
            P1OUT &= ~BIT0; __delay_cycles(100000);
        }
    }

    while(1) {
        currentRead = readTemperature();

        if (enableWrite) {
            writeLastTemp(currentRead);
```

```
        }


        if (recovered > currentRead) {
            P1OUT |= BIT0;
        } else {
            P1OUT &= ~BIT0;
        }


        if (!(P1IN & BIT2)) {
            enableWrite = 1;
        }


        __delay_cycles(100000);
    }
}
```